

ShopEZ

E-Commerce Application

Project Documentation Report

Full Stack Development with MERN

Developer	Dhanish Ladwani
GitHub	https://github.com/dhanish0711/ShopEZ-E-Commerce-Application
Demo Video	https://drive.google.com/file/d/1GPxt3Ikb-l6m5CMUCZQViXLTWxwsJrn/view?usp=sharing
Technology Stack	React, Node.js, Express.js, MongoDB
Version	2.0.0

1. Introduction

This document provides comprehensive technical documentation for ShopEZ, a production-ready full-stack e-commerce application built on the MERN stack (MongoDB, Express.js, React, Node.js).

Field	Details
Project Title	ShopEZ — E-Commerce Application
Developer / Author	Dhanish Ladwani
GitHub Profile	github.com/dhanish0711
Repository	github.com/dhanish0711/SHOPEZ--E-commerce-Application
Technology Stack	MERN (MongoDB, Express.js, React 19, Node.js 18+)
API Version	v2.0.0
License	MIT Open Source License

Roles

Team Member	Role	Responsibility
Dhanish Ladwani	Full Stack Developer	Frontend (React/Vite), Backend (Node/Express), Database (MongoDB), UI/UX Design

2. Project Overview

Purpose

ShopEZ is a robust, production-grade e-commerce platform designed to deliver a seamless online shopping experience. The project demonstrates expertise across the entire MERN stack by implementing a feature-rich storefront for customers and a comprehensive control panel for administrators. The application is engineered with a focus on security, performance, and scalability.

Key Features

Customer Experience

- Secure JWT-based authentication with bcryptjs password hashing and persistent sessions stored in localStorage
- Intuitive product catalog with real-time text search, gender and category filters, and multi-option sorting (price / discount)
- Star ratings and user review counts displayed on all product listings
- Dynamic shopping cart with backend synchronization, quantity adjustment, and instant order summary updates
- Order tracking with full order history, delivery status, and estimated delivery times
- Wishlist functionality allowing customers to toggle and save favourite products

Admin Control Panel

- Insightful analytics dashboard with real-time charts: monthly revenue, order distribution, user registration stats, and best-selling products
- Product builder with dynamic form and live component preview for publishing new listings
- Order fulfillment manager with status controls: order placed, processing, shipped, delivered, cancelled
- User auditing panel to view all customer profiles and manage registrations

Security & Performance

- HTTP header hardening via the Helmet middleware
- Rate limiting: 30 requests per 15 minutes on auth endpoints; 300 requests per 15 minutes on general API routes
- Response compression middleware to reduce payload sizes and improve load times
- Strictly configured CORS to allow only trusted client origins

3. Architecture

Frontend (React + Vite SPA)

The client is a Single Page Application built with React 19 and bundled using Vite 8. It follows a component-based architecture with a clean separation between reusable UI components, page-level views, and global state management.

- React Router v7 handles client-side routing with PrivateRoute and AdminRoute guard components
- React Context API (AuthContext) manages global authentication state, persisting userInfo to localStorage
- Axios is configured with a base URL and automatically attaches the JWT token via a request interceptor
- All styling is implemented in pure vanilla CSS using custom properties for a bespoke design system

Backend (Node.js + Express.js)

The server follows the MVC (Model-View-Controller) pattern, separating concerns into routes, controllers, models, and middleware. Express.js 5 powers the REST API.

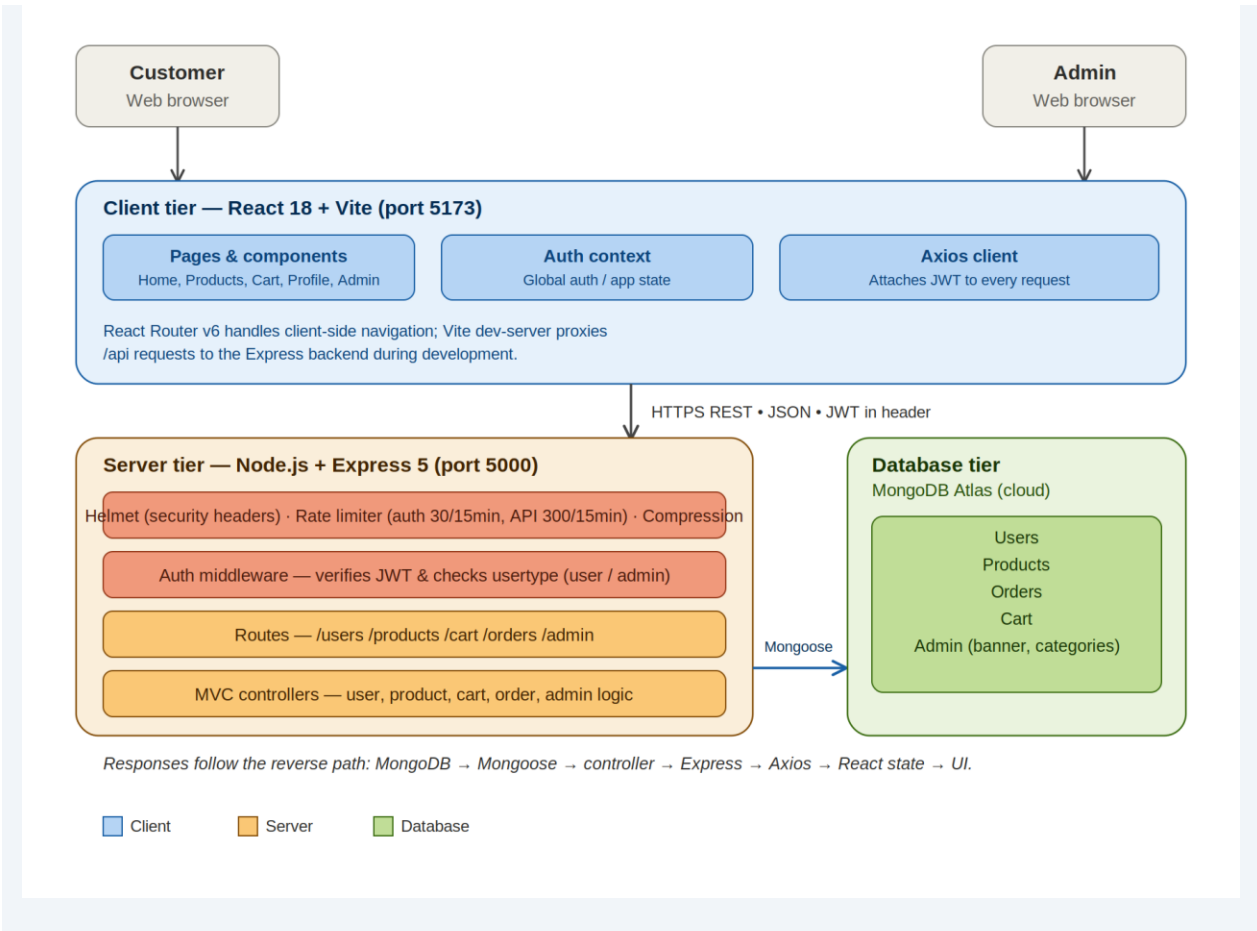
- server.js is the entry point, wiring together all middleware and route handlers
- Security stack: Helmet, express-rate-limit, CORS, and bcryptjs
- HTTP request logging via Morgan in development mode
- Response body compression via the Compression middleware
- A global error handler and a 404 catch-all ensure consistent JSON error responses

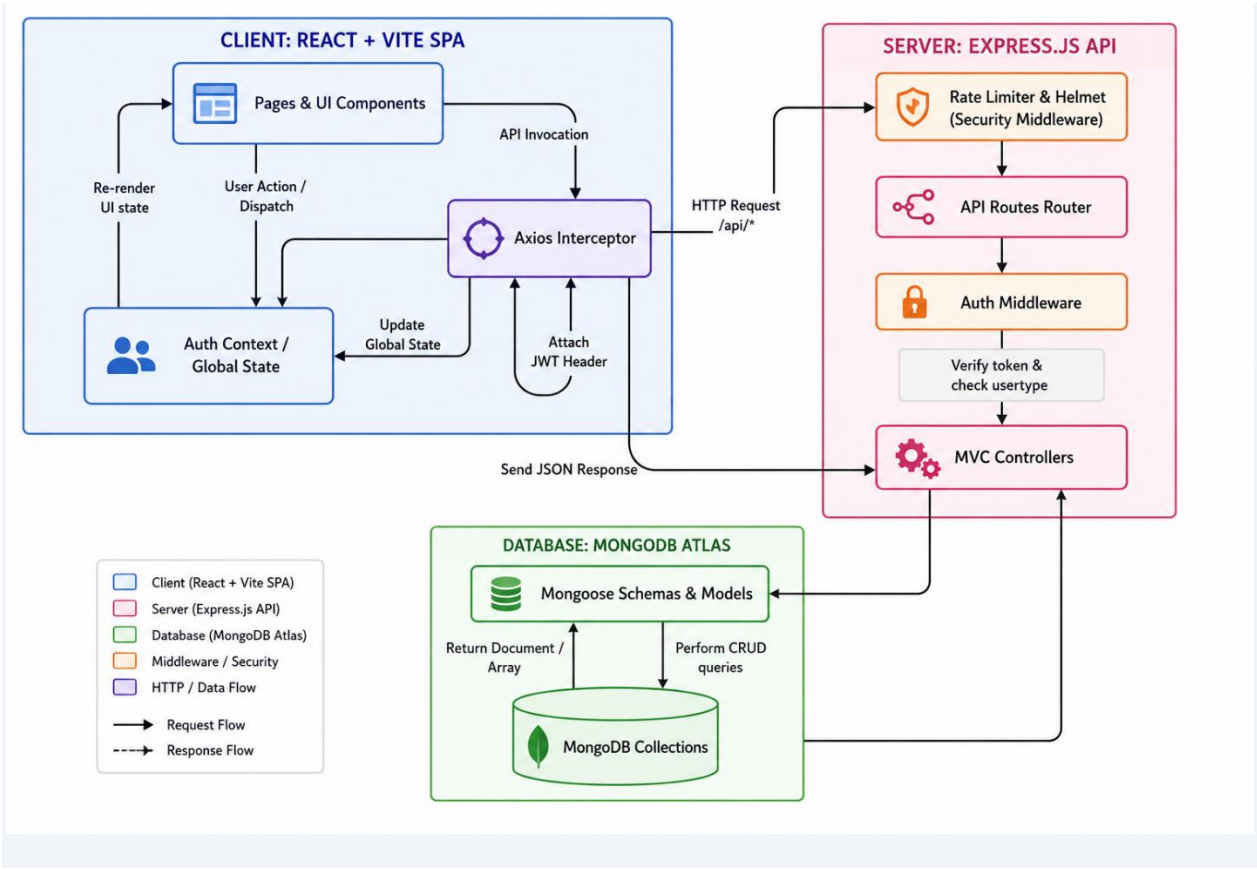
Database (MongoDB + Mongoose)

MongoDB Atlas (cloud-hosted NoSQL database) is used as the primary data store. Mongoose provides schema definition, validation, and an ODM layer for all database interactions.

Collections: users, products, orders, cart, admin. All schemas are defined centrally in Server/models/Schema.js.

System Architecture Diagram





4. Setup Instructions

Prerequisites

Software	Minimum Version	Purpose
Node.js	v18.0.0 LTS+	JavaScript runtime for the server and build tools
npm	v9.0.0+	Package manager (bundled with Node.js)
MongoDB Atlas	Shared M0 Cluster+	Cloud-hosted NoSQL database (or local MongoDB v6.0+)
Git	v2.30.0+	Source control for cloning the repository

Hardware Requirements

Specification	Minimum	Recommended
CPU	Dual-Core 2.0 GHz	Quad-Core 2.5 GHz+
RAM	4 GB	8 GB (for concurrent dev servers)
Disk Space	500 MB free	1 GB free

Specification	Minimum	Recommended
Network	Internet connection required	Stable broadband for MongoDB Atlas

Installation Steps

Step 1 — Clone the Repository

```
git clone https://github.com/dhanish0711/SHOPEZ--E-commerce-Application.git
cd "SHOPEZ E-commerce Application"
```

Step 2 — Install All Dependencies

```
npm run install:all
```

Step 3 — Configure Environment Variables

```
cp Server/.env.example Server/.env
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/shop
ez
JWT_SECRET=your_super_secret_jwt_key_change_this_in_production
NODE_ENV=development
```

Step 4 — Seed the Database

```
npm run seed
```

Step 5 — Launch the Application

```
npm run dev
```

5. Folder Structure

Client (React + Vite Frontend)

Path	Description
Client/src/components/	Reusable UI elements (Navbar, Footer, ProductCard, AdminLayout)
Client/src/context/AuthContext.jsx	Global authentication state manager using React Context API
Client/src/pages/	Page-level views (Home, Products, Login, Register, Cart, Profile, OrderDetails)
Client/src/pages/admin/	Admin-only views (AdminDashboard, AllOrders, AllProducts, NewProduct)
Client/src/utils/api.js	Axios instance configured with base URL and JWT interceptor
Client/src/index.css	Global CSS: design tokens (CSS variables), layout styles, responsive rules
Client/src/App.jsx	Root router definition; includes PrivateRoute and AdminRoute guards

Path	Description
Client/src/main.jsx	React application entry point; wraps app in AuthProvider
Client/vite.config.js	Vite configuration with dev-server API proxy to localhost:5000
Client/public/	Static public assets (favicon.svg, icons.svg)

Server (Node.js + Express Backend)

Path	Description
Server/server.js	Express app entry point: middleware wiring, route registration, error handlers
Server/config/db.js	MongoDB connection logic using Mongoose
Server/models/Schema.js	Centralized Mongoose schemas: User, Admin, Product, Order, Cart
Server/controllers/	MVC controllers: adminController, cartController, orderController, productController, userController
Server/middleware/authMiddleware.js	JWT token verification and admin privilege validation
Server/routes/	Express routers: adminRoutes, cartRoutes, orderRoutes, productRoutes, userRoutes
Server/seed.js	Database seeder script to populate demo data
Server/.env.example	Environment variable template

6. Running the Application

Development Mode (Both Servers Concurrently)

```
# From the project root directory
npm run dev
```

Individual Servers

```
# Frontend only (from Client/ directory)
cd Client && npm run dev

# Backend only (from Server/ directory)
cd Server && npm run dev    # nodemon hot-reload
cd Server && npm start      # production (node server.js)
```

Access URLs

Service	URL	Description
Frontend Client	http://localhost:5173	React + Vite SPA (HMR enabled)
Backend API	http://localhost:5000	Express REST API

Service	URL	Description
Health Check	http://localhost:5000/api/health	Server status, version, uptime

Port Requirements

Port	Service	Notes
5173	React Client (Vite)	Default Vite dev server port
5000	Express Server	Configurable via PORT in .env
27017	MongoDB Local	Only if not using MongoDB Atlas

7. API Documentation

Base API URL: http://localhost:5000/api

All protected endpoints require the Authorization header: Bearer <JWT_TOKEN>

Authentication & User Endpoints

Method	Endpoint	Auth	Description
POST	/users/register	Public	Register a new user account
POST	/users/login	Public	Authenticate credentials and return JWT token
GET	/users/profile	User	Retrieve authenticated user profile
PUT	/users/profile	User	Update user profile details or password
GET	/users	Admin	List all registered user accounts
DELETE	/users/:id	Admin	Delete a user account by MongoDB ID

Product Endpoints

Method	Endpoint	Auth	Description
GET	/products	Public	List all products. Supports: search, gender, category, sort, minPrice, maxPrice
GET	/products/categories	Public	Retrieve list of distinct product categories
GET	/products/:id	Public	Get full details of a single product by ID
POST	/products	Admin	Create and publish a new product listing
PUT	/products/:id	Admin	Update an existing product details
DELETE	/products/:id	Admin	Remove a product from the catalog

Cart Endpoints

Method	Endpoint	Auth	Description
GET	/cart	User	Retrieve all items in the authenticated user cart
POST	/cart	User	Add a product item to the user cart
PUT	/cart/:id	User	Update the quantity of a specific cart item
DELETE	/cart/:id	User	Remove a specific item from the cart
DELETE	/cart/clear	User	Clear all items from the user cart

Order Endpoints

Method	Endpoint	Auth	Description
POST	/orders	User	Submit and place a new order
GET	/orders/myorders	User	List all orders placed by the current user
GET	/orders/:id	User	Get full details of a specific order by ID
GET	/orders	Admin	List all orders across all users in the system
PUT	/orders/:id/status	Admin	Update the fulfillment status of an order
DELETE	/orders/:id	Admin	Permanently delete an order record

Admin Endpoints

Method	Endpoint	Auth	Description
GET	/admin	Public	Homepage data: banner image and category list
GET	/admin/dashboard	Admin	Store analytics: revenue charts, top products, user stats
PUT	/admin/banner	Admin	Update the store homepage banner image URL
PUT	/admin/categories	Admin	Update the allowed product categories list

8. Authentication

Authentication Strategy

ShopEZ uses a stateless JWT (JSON Web Token) authentication strategy. No server-side sessions are maintained; all identity information is encoded in the token itself.

Registration & Login Flow

- User submits credentials to POST /api/users/register or /api/users/login
- Passwords are hashed using bcryptjs with a salt factor of 10 before storage

- On successful authentication, the server returns a signed JWT valid for 30 days
- The client stores the full user object (including the token) in localStorage via AuthContext
- The Axios instance attaches the token as a Bearer token in the Authorization header on every subsequent request

Server-Side Token Verification

- Extracts the Bearer token from the Authorization request header
- Verifies the token signature using the JWT_SECRET environment variable
- Loads the corresponding user document from MongoDB and attaches it to req.user
- Checks req.user.usertype === admin to enforce admin-only route access
- Returns 401 Unauthorized or 403 Forbidden JSON responses on failure

Client-Side Route Guards

Guard Component	Condition	Redirect Behaviour
PrivateRoute	userInfo exists in AuthContext	Redirects to /login if unauthenticated
AdminRoute	userInfo.usertype === admin	Redirects to /login or / if not an admin

Rate Limiting on Auth Endpoints

Scope	Limit	Window
POST /api/users/login	30 requests	Per 15 minutes, per IP
POST /api/users/register	30 requests	Per 15 minutes, per IP
All /api/* routes	300 requests	Per 15 minutes, per IP

9. User Interface

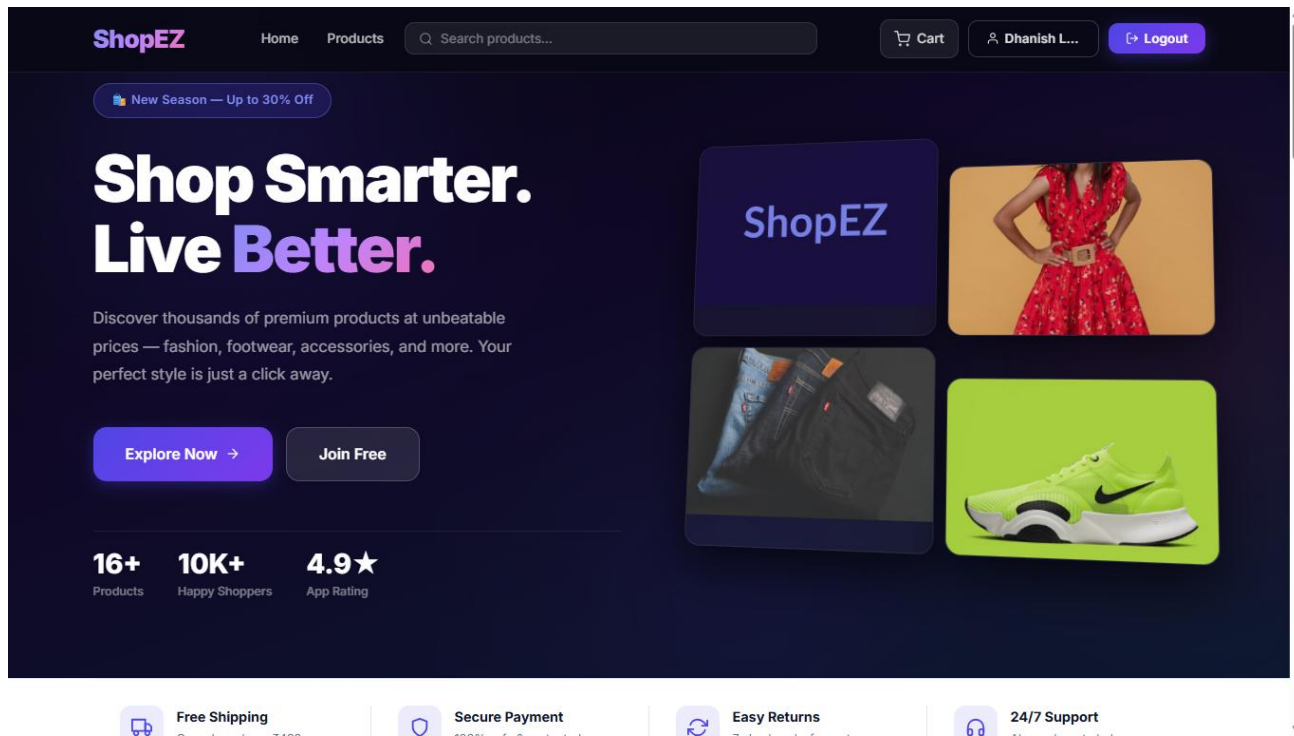
Design Philosophy

The entire UI is built from scratch using pure vanilla CSS with CSS custom properties (variables). No external UI frameworks are used, resulting in a lightweight, custom-tailored design system.

Application Pages & Screenshots

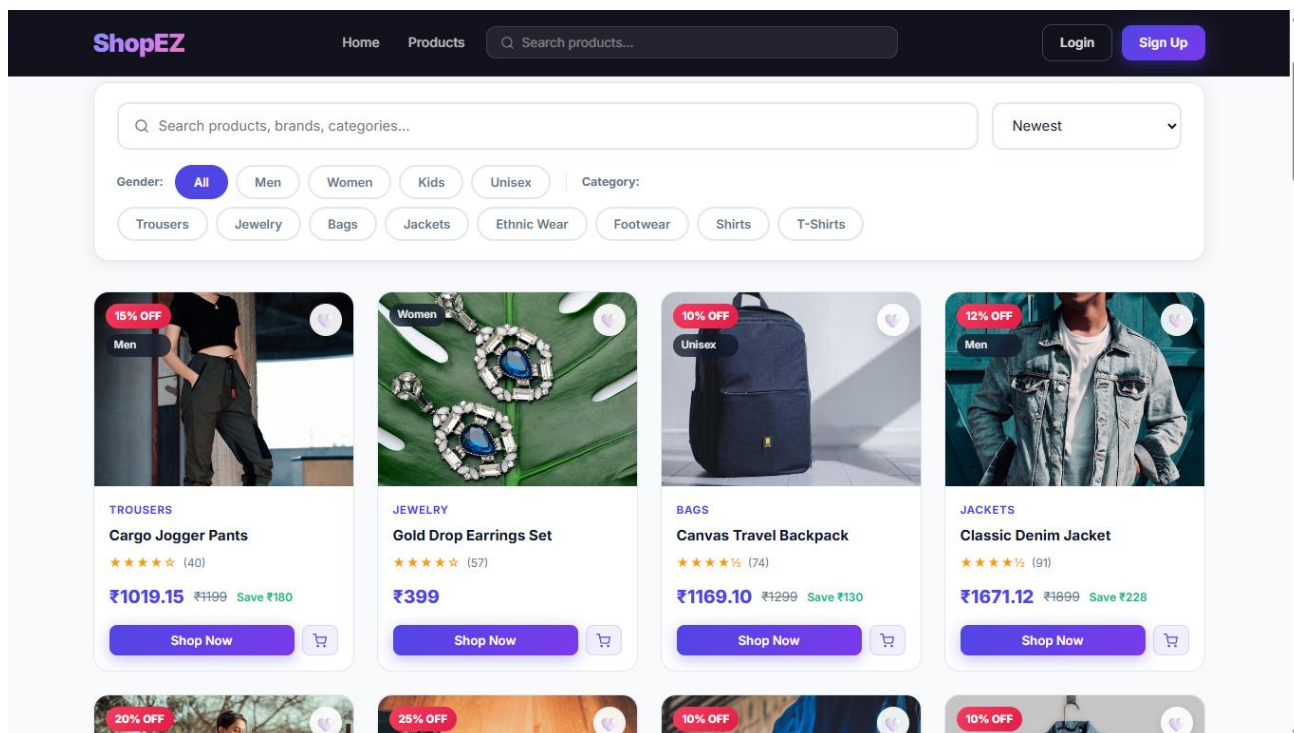
Home Page (/)

Hero section with featured banners, promotional highlights, and product category navigation.



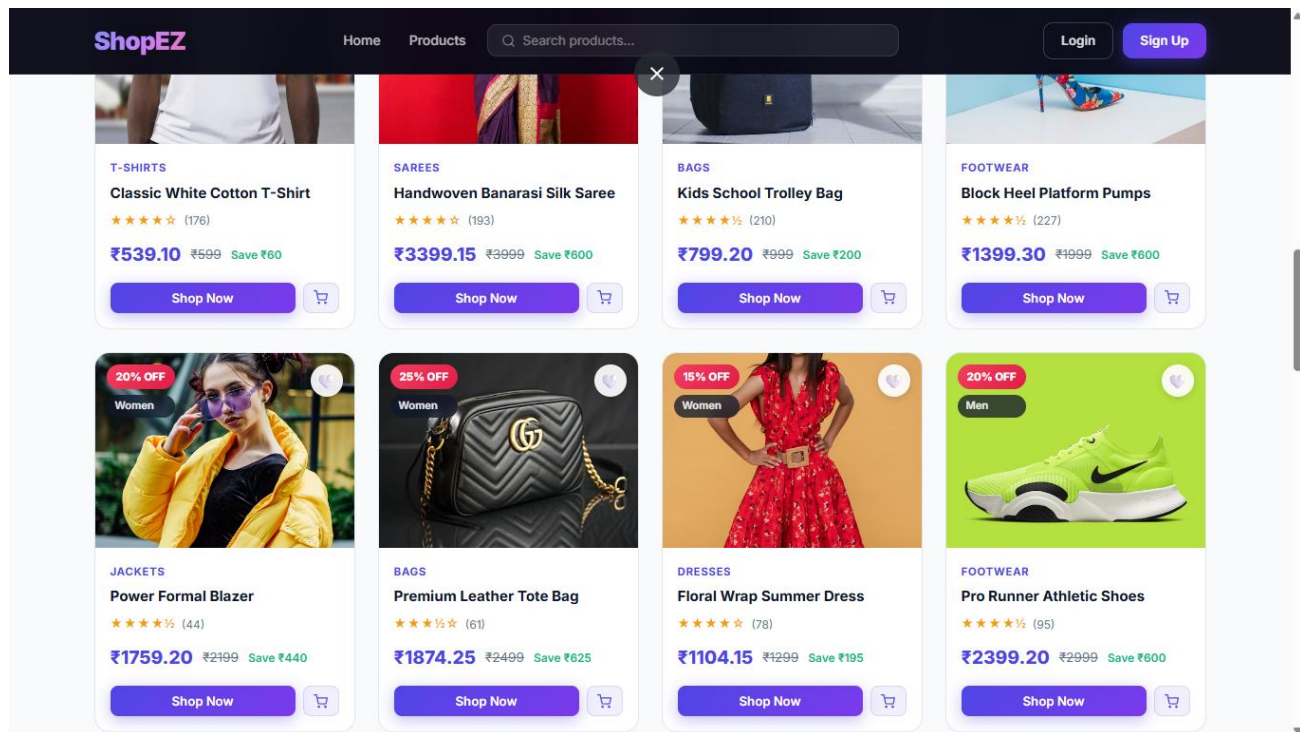
Products Catalog (/products)

Full product grid with live text search, gender and category filter sidebar, and price / discount sort controls.



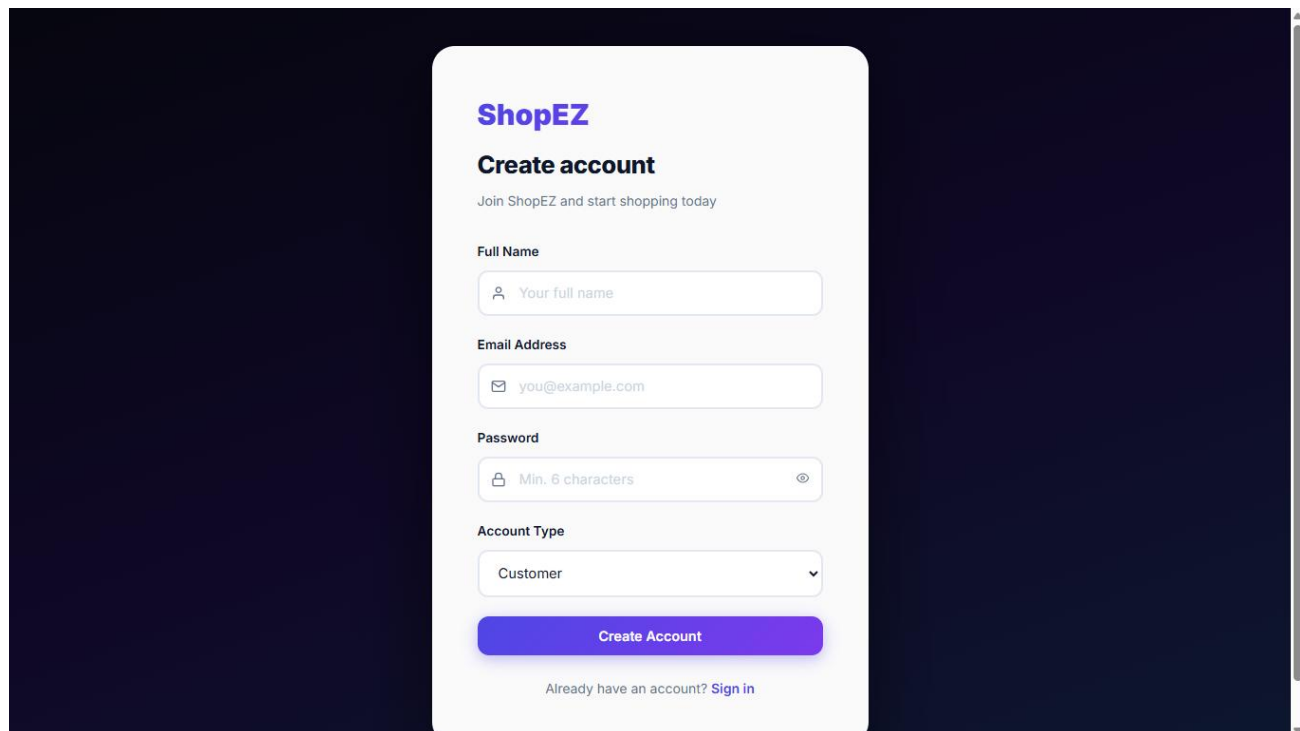
Product Card Component

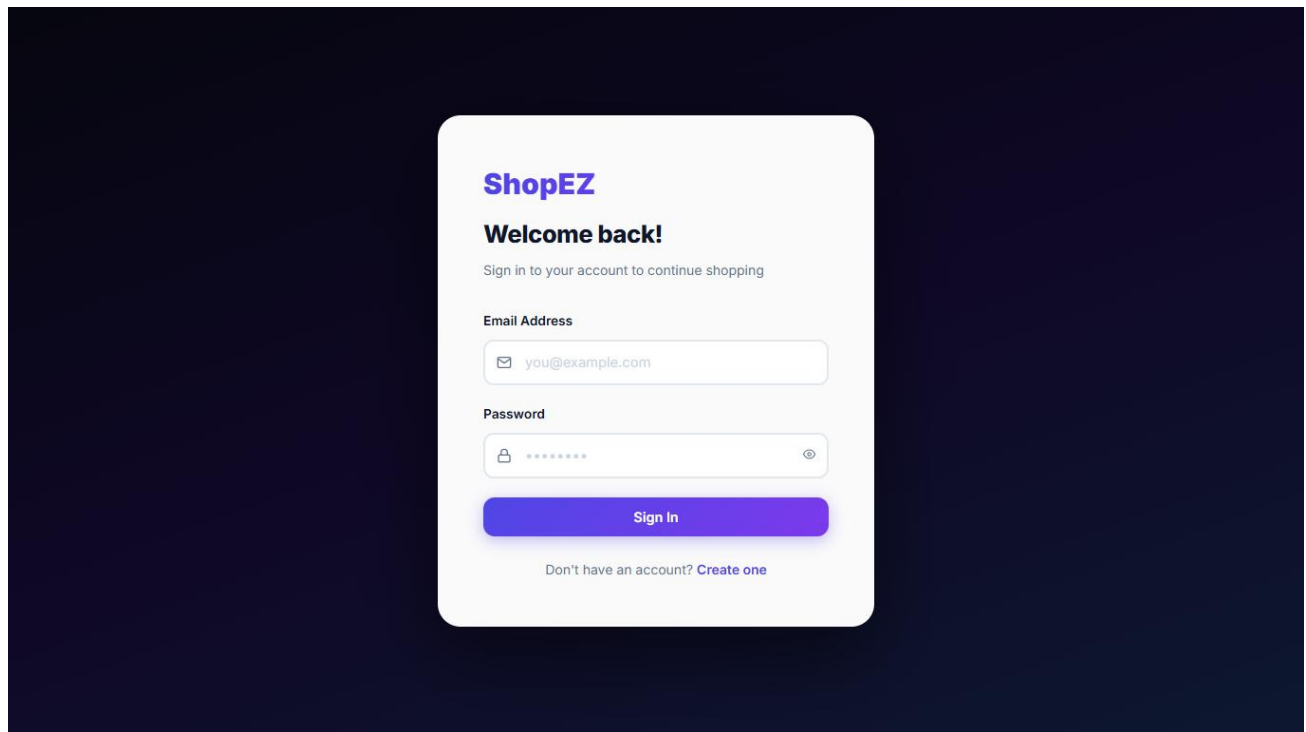
Interactive product card displaying image, title, star rating, original price, discounted price, size selector, and wishlist toggle.



Login & Register Pages (/login | /register)

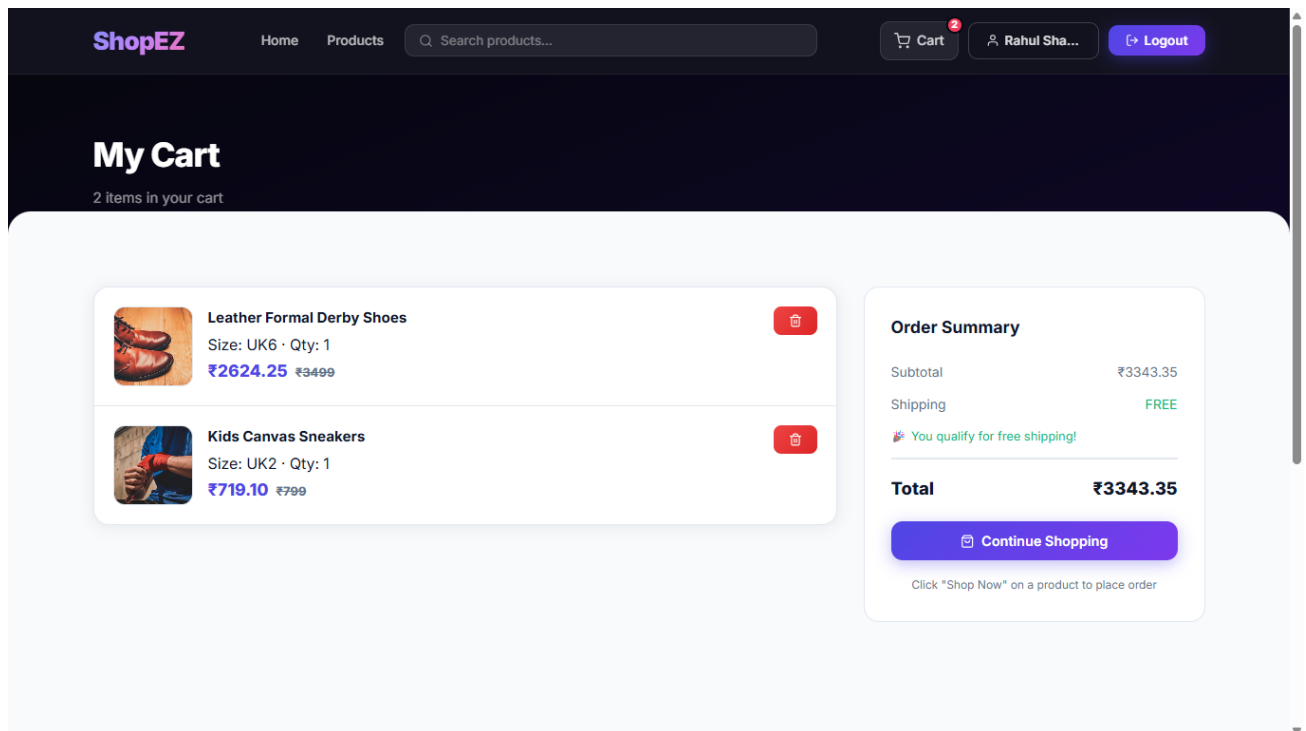
User authentication forms with input validation and error messaging.





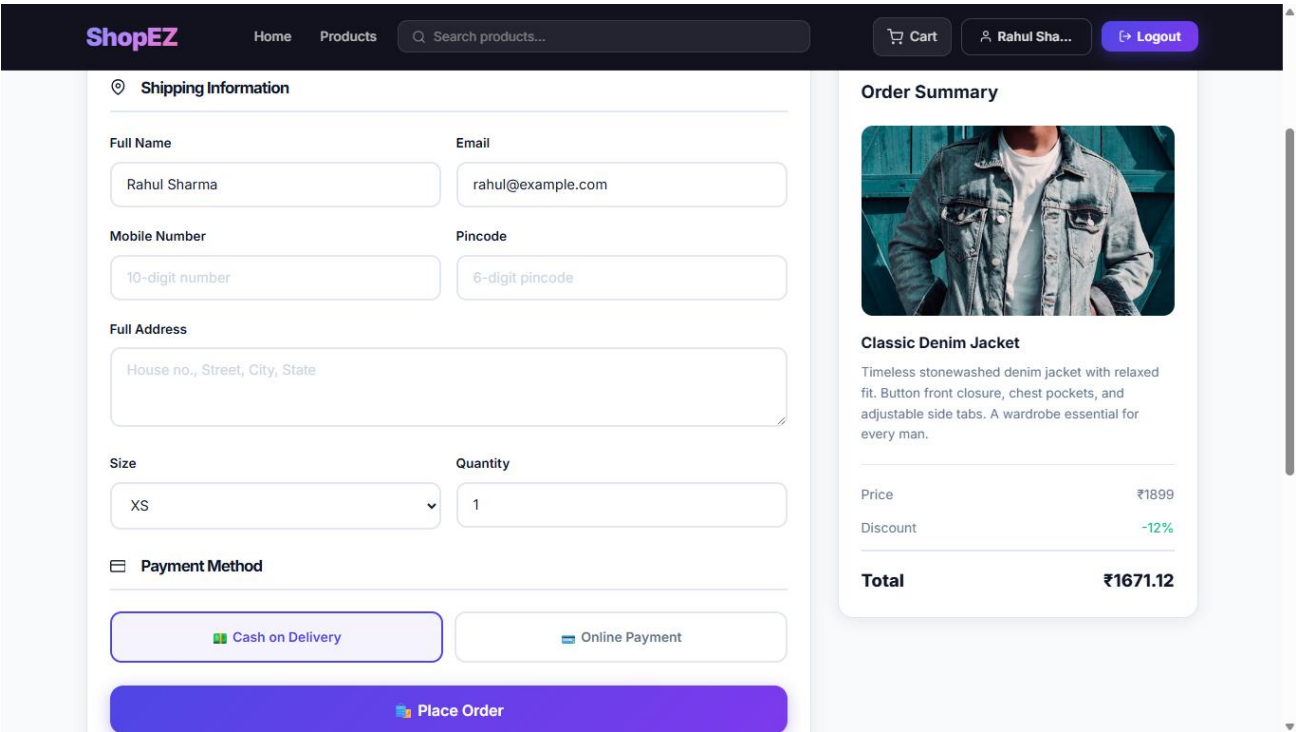
Shopping Cart (/cart)

Cart page displaying all items, quantity adjustment controls, per-item totals, cart summary, and checkout button.



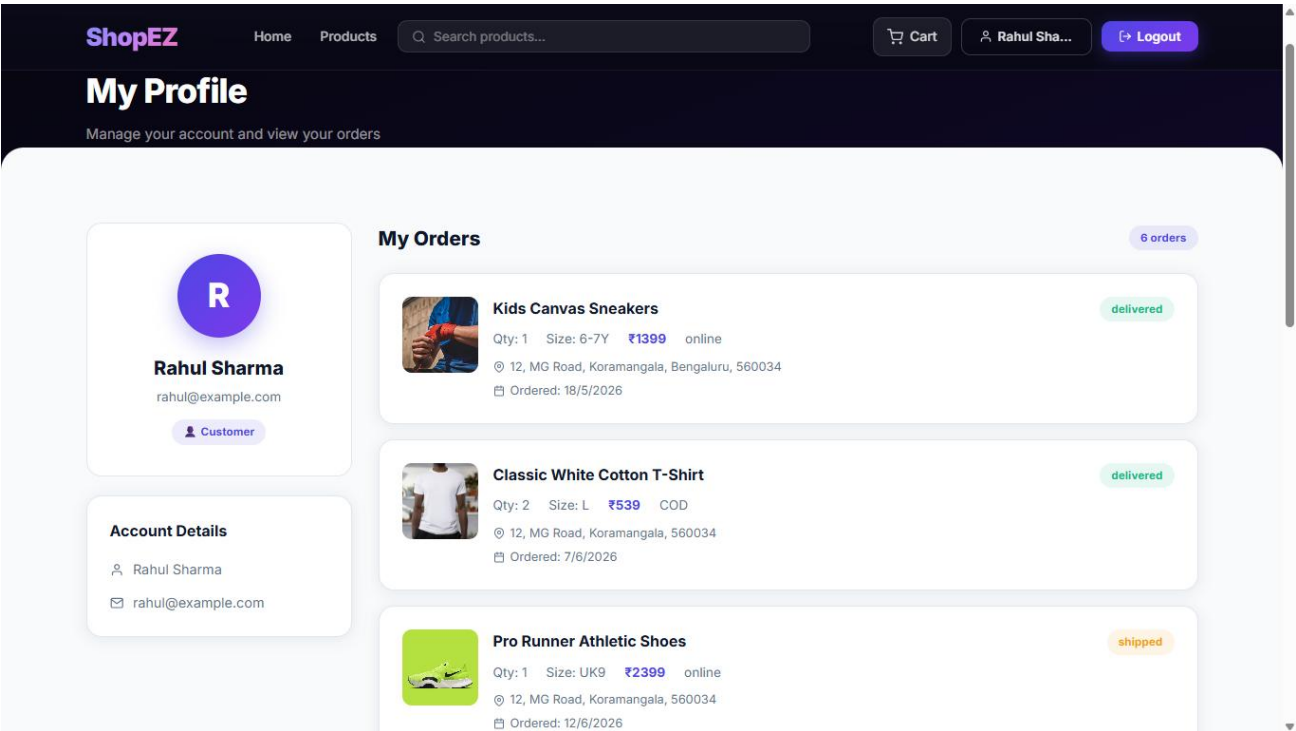
Order Details (/order/:id)

Checkout flow with shipping address form, payment method selection, and order confirmation summary.



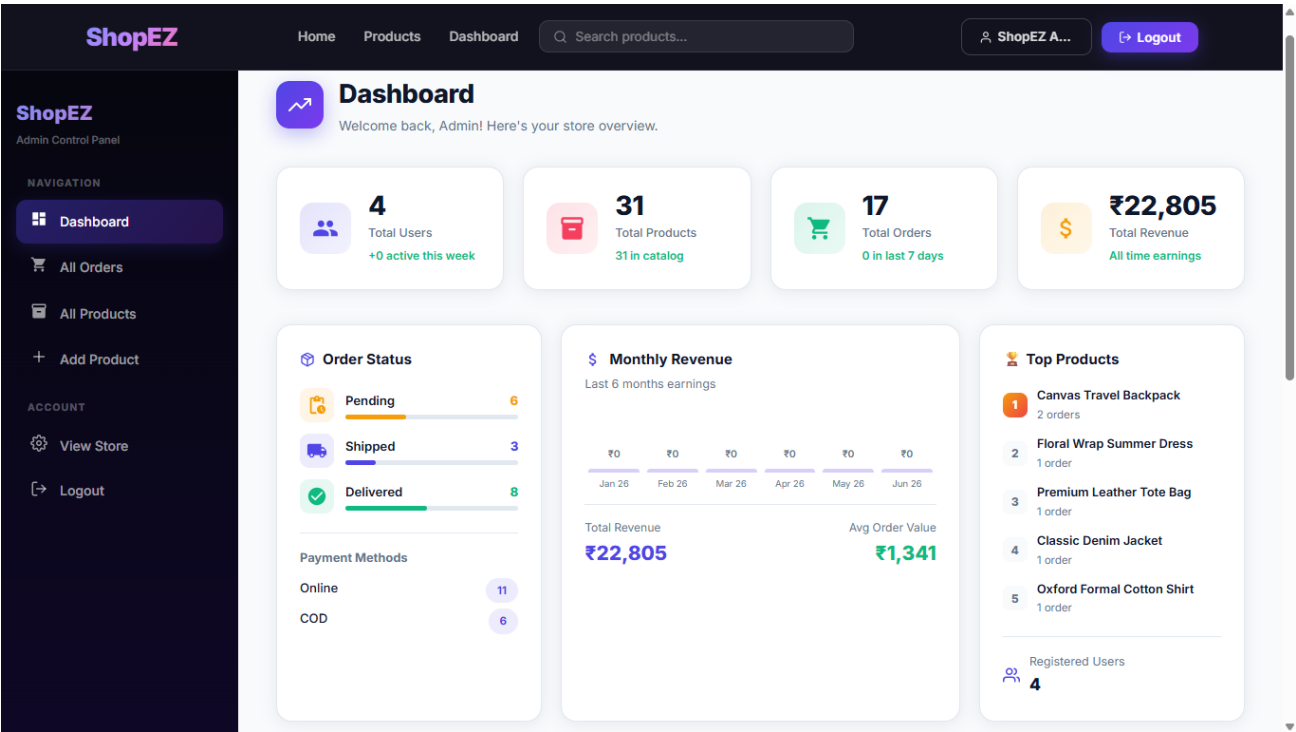
User Profile (/profile)

Editable user profile section and a scrollable list of past orders with status badges.



Admin Dashboard (/admin)

Real-time analytics dashboard with monthly revenue bar chart, order status donut chart, user registration line chart, and top-selling products table.



Admin Order Management (/admin/orders)

Full order list with status controls (order placed, processing, shipped, delivered, cancelled) and delete option.

All Orders								
17 total orders								
PRODUCT	CUSTOMER	CONTACT	ADDRESS	QTY	AMOUNT	PAYMENT	DATE	STATUS
 Smart Fitness Tra... Size: S/M	Arjun Mehta	arjun@example.com 9988776655	78, Connaught Place, New Delhi, 110001	1	₹1199	online	13/6/2026	delivered
 Canvas Travel Bac... Size: One Size	Rahul Sharma	rahul@example.com 7894561234	house 1 mount road sadar nagpur, 440012	1	₹1169.1	online	6/18/2026	order placed
 Kids Waterproof R... Size: 7-8Y	Rahul Sharma	rahul@example.com 7894561237	Flat No-101 Manohar Apartments Civil Lines Nagpur, 440001	1	₹1039.2	online	6/17/2026	processing
 Power Formal Blazer Size: S	Priya Patel	priya@example.com 9123456789	45, Bandra West, Mumbai, 400050	1	₹1759	online	15/6/2026	order placed

Admin Product Management (/admin/products)

Full product catalog manager with edit and delete options for each listing.

The screenshot displays the 'All Products' page in the ShopEZ admin dashboard. The left sidebar contains navigation links: Dashboard, All Orders, All Products (selected), Add Product, View Store, and Logout. The main content area shows a table of 31 products in the catalog. The table has columns for Product, Category, Gender, Sizes, Price, Discount, Final Price, and Actions. Each product row includes a product image, title, category, gender, sizes, price, discount percentage, final price, and a delete button.

PRODUCT	CATEGORY	GENDER	SIZES	PRICE	DISCOUNT	FINAL PRICE	ACTIONS
 Cargo Jogger Pants Versatile cargo jogger pants ...	Trousers	Men	S, M, L, XL, XXL	₹1199	15%	₹1019.15	Delete
 Gold Drop Earrings Set Elegant gold-plated drop earr...	Jewelry	Women	One Size	₹399	—	₹399	Delete
 Canvas Travel Backpack Spacious 30L canvas backpa...	Bags	Unisex	One Size	₹1299	10%	₹1169.10	Delete
 Classic Denim Jacket Timeless stonewashed denim...	Jackets	Men	XS, S, M, L, XL	₹1899	12%	₹1671.12	Delete
 Embroidered Anarkali K... Hand-embroidered Anarkali K...	Ethnic Wear	Women	XS, S, M, L, XL	₹1599	20%	₹1279.20	Delete
 Leather Formal Derby S... Premium genuine leather der...	Footwear	Men	UK6, UK7, UK8	₹3499	25%	₹2624.25	Delete

New Product Form (/admin/products/new)

Dynamic product creation form with a real-time live preview of the published product card.

The screenshot displays the 'Add New Product' page in the ShopEZ admin dashboard. The left sidebar is identical to the previous page. The main content area is titled 'Add New Product' and includes a sub-header 'Fill in the details to add a product to the catalog'. The form is divided into two main sections: 'Basic Information' and 'Live Preview'. The 'Basic Information' section contains fields for Product Title, Description, Category, Gender, Price, and Discount. The 'Live Preview' section shows a real-time preview of the product card, including the category, product title, and price.

Basic Information

Product Title *
e.g. Classic Cotton T-Shirt

Description *
Describe the product in detail...

Category *
e.g. T-Shirts, Jeans, Shoes

Gender *
Men

Price (₹) *
0.00

Discount (%)
0

Images

Live Preview

No image URL yet

CATEGORY - MEN
Product Title
₹0

Reusable Components

Component	Description
Navbar.jsx	Top navigation bar with conditional links based on auth state and user role
Footer.jsx	Site-wide footer with navigation links and branding
ProductCard.jsx	Interactive card with wishlist toggle, star rating, discount badge, and size selector
AdminLayout.jsx	Admin section wrapper providing the sidebar navigation and header for all admin views

10. Testing

Testing Strategy

The current version of ShopEZ does not include an automated test suite. Testing has been performed manually to verify all user flows, API endpoints, and edge-case error handling.

Manual Testing Areas

Area	Test Cases Covered
Authentication	Register new user, login with correct/incorrect credentials, JWT expiry behaviour, admin vs user role access
Product Catalog	Search by keyword, filter by gender and category, sort by price and discount, view product details
Cart Management	Add items, adjust quantities, remove individual items, clear entire cart, cart persists on re-login
Order Flow	Place order, view order in profile history, view order details page
Admin Dashboard	Login as admin, view analytics, add new product, update order status, delete product
Rate Limiting	Exceed auth rate limit threshold and verify 429 error response
API Health	GET /api/health returns expected JSON with server metadata

Recommended Future Testing Tools

- Jest — JavaScript unit testing framework for controllers and utility functions
- Supertest — HTTP assertion library for integration testing Express API endpoints
- React Testing Library — Component-level testing for React UI elements
- Postman / Thunder Client — API endpoint manual testing and collection documentation

11. Demo

Demo Accounts

Run `npm run seed` from the project root to populate these demo credentials:

Role	Email	Password	Access Level
Store Admin	admin@shopez.com	admin123	Full admin panel access
Customer 1	rahul@example.com	user123	Standard shopping access
Customer 2	priya@example.com	user123	Standard shopping access
Customer 3	arjun@example.com	user123	Standard shopping access

Application Demo Video Link

<https://drive.google.com/file/d/1GPxt3Ikb-l6m5CMUCZQViXLTWxwsJjrj/view>

12. Known Issues

#	Issue	Area	Severity
1	No automated test suite is present; all testing is manual	Testing	Low
2	Images are stored as external URLs; no file upload support is implemented	Products	Medium
3	Cart quantity is stored as a String type in the Cart schema, requiring parsing when computing totals	Database	Low
4	No email verification is enforced on user registration	Auth	Low
5	The seed.js script may fail if the database already contains existing data (no upsert logic)	DevOps	Low
6	No pagination is implemented on the product catalog or admin order/product lists	Performance	Medium
7	Admin credentials can be set via the usertype field at registration with no secondary confirmation	Security	Medium

13. Future Enhancements

Planned Improvements

Enhancement	Priority	Description
Automated Testing	High	Add Jest + Supertest for backend unit/integration tests; React Testing Library for UI tests
Pagination & Infinite Scroll	High	Implement cursor-based pagination on products and admin lists to handle large catalogs

Enhancement	Priority	Description
Payment Gateway Integration	High	Integrate Razorpay or Stripe for real payment processing and transaction records
Image Upload	Medium	Add Cloudinary or AWS S3 integration for direct product image file uploads from the admin panel
Email Notifications	Medium	Send transactional emails (order confirmation, shipping updates) via Nodemailer or SendGrid
Email Verification	Medium	Enforce email verification on user registration using time-limited tokens
Product Reviews & Ratings	Medium	Allow customers to submit star ratings and written reviews; aggregate on product listings
Search Autocomplete	Medium	Implement Elasticsearch or MongoDB Atlas Search for fast, typo-tolerant product search
Wishlist Persistence	Low	Save wishlist state to the database so it persists across devices and sessions
Refresh Token Rotation	Low	Implement short-lived access tokens and long-lived refresh tokens for better security
PWA Support	Low	Add a Service Worker and Web App Manifest to enable offline browsing and mobile install
Discount Coupons	Low	Admin-configurable discount coupon codes that apply at cart checkout

End of Documentation